



1. Contexte du projet

Le projet réalisé dans le cadre de cette SAE consiste à développer une application web de suivi de commande de colis au sein d'une université. L'application met en relation plusieurs acteurs :

- Les départements créent des demandes d'achat via des devis qui sont ensuite vérifier
- Le service financier contrôle les budgets et valide ou rejette les devis
- Le directeur signe les devis validés afin de générer des bons de commande
- Le service postal universitaire réceptionne et transfère les colis
- Le service postal IUT réceptionne les colis, les remet aux destinataires et assure le suivi final
- L'administrateur gère le site internet, si y a un problème de connexion, c'est lui qui s'en occupe.

L'objectif principal du projet est d'avoir la traçabilité complète d'un colis, depuis la demande d'achat jusqu'à la remise finale au département.

2. Architecture générale et choix techniques

L'application est sur une architecture MVC (Modèle - Vue - Contrôleur), permettant une séparation claire des responsabilités :

- **Modèles** : gestion de la base de données et des requêtes SQL
- **Contrôleurs** : Ici c'est toute la logique, validation des actions et transitions de statuts
- **Vues** : affichage dynamique des données selon le rôle utilisateur

models	40
DépartementMod...	41
DirecteurModels...	42
DirecteurModels...	43

controllers	2
DépartementCont...	2
DirecteurControll...	2

views	38
> departement	39
> directeur-iut	40
> finance	41
> finance	42

✓ Avantage ✓

- Le code est facile à lire et à comprendre, (Donc même après plusieurs semaine on pourra toujours comprendre les code car ils sont très bien structuré

- L'architecture MVC qui est dans ce projet facilite grandement la maintenance de l'application (Modifier ou corriger une partie du projet sans casser le reste.

- Il y a une grande séparation claire des responsabilités (Chaque partie du code a un rôle précis donc personne ne fait le travail de l'autre



Grâce au MVC :

- Les requêtes SQL sont dans les Models
- Les actions utilisateur sont dans les Controllers
- L'affichage HTML est dans les Views

Ce modèle MVC nous a été recommander par le responsable de la SAE en lui même. Cela nécessitait pas de mise à jour donc c'était parfait selon lui



Exemples :

Si on veut changer l'affichage d'un statut d'un colis :

"Retiré" → "Livré au destinataire"

On a juste à aller dans le view et changer
Retiré



Livré au destinataire



Les responsabilité sont :

Le Model :

- Requêtes SQL
- Connexion à la base
- Règles liées aux données

Le Contrôleur :

- décider quoi faire
- appeler les bons modèles
- choisir la bonne vue

La View

- HTML , CSS , JS
- affichage des donnée

3. Optimisation des algorithmes

Les requêtes SQL ont été pensées pour être :

- ciblées (on prend uniquement des colonnes nécessaires)
- regroupées lorsque possible (COUNT, JOIN, etc.)

Bénéfice : réduction du nombre de requêtes et amélioration des performances globale de notre application

4. Sécurité de l'application

4.1 Authentification CAS universitaire

CasAuthenticator.php
CasConfiguration.php

L'application repose sur une authentification CAS (Central Authentication Service) qui est un système officiel de l'université.

Ce choix permet :

- d'éviter de mettre des mots de passe local
- de ne jamais stocker de mot de passe en base de données
- de s'appuyer sur un service sécurisé et éprouvé officiellement

4.2 Middleware d'authentification

Un middleware d'authentification a été mis en place afin de :

- vérifier qu'un utilisateur est bien authentifié avant chaque accès
- stocker l'utilisateur en session après la première authentification
- éviter des appels CAS répétés (optimisation)

4.3 Middleware autorisations par rôle

Un second middleware gère les droits d'accès selon le rôle :

- chaque rôle dispose de routes autorisées
- toute tentative d'accès non autorisée est bloquée côté serveur

4.4 Requêtes préparées

Toutes les requêtes SQL utilisent des requêtes préparées via PDO, ce qui permet de se protéger contre les injections SQL.

Par exemple :

```
44 public function getBonsCommandeRecents(): mixed {  
45     return $this->db->query("  
46         SELECT numero_commande, date_commande, montant_estime, statut  
47         FROM bon_commande  
48         ORDER BY date_commande DESC  
49         LIMIT 10  
50     ")>fetchAll(PDO::FETCH_ASSOC);  
51 }
```

```
1 reference | 0 overrides  
89 public function rejeterDevis($id): void {  
90     $sql = "UPDATE devis SET statut = 'rejete_finance' WHERE id_devis = ?";  
91     $req = $this->db->prepare($sql);  
92     $req->execute([$id]);  
93 }
```

5. Idée et amélioration possible

◆ Mise en cache des données souvent utilisées :

Actuellement, certaines chiffres comme (nombre de colis, devis en attente, bons de commande, etc.) sont recalculés à chaque chargement de page via des requêtes SQL.

– Amélioration qu'on pourrait faire :

- mettre en place un cache (fichier temporaire)
- recalcul uniquement lors d'un M.A.J (nouveau colis, changement de statut etc...)

✓ Avantage :

réduction des accès à la base de données
amélioration des temps de chargement

Exemples :

Utilisation de COUNT(*) directement en base pour les statistiques

Jointures SQL pour éviter des boucles PHP coûteuses

Utilisation d'index sur : numero_commande , numero_suivi , departement_id , statut_id

```
1 reference | 0 overrides  
public function countBonCommande(): mixed {  
    return $this->db  
        ->query("SELECT COUNT(*) FROM bon_commande")  
        ->fetchColumn(PDO::FETCH_ASSOC);  
}  
  
1 reference | 0 overrides  
public function getBudgetsDepartements(): mixed {  
    return $this->db->query("  
        SELECT nom, budget_total, budget_utilise  
        FROM departement  
    ")>fetchAll(PDO::FETCH_ASSOC);  
}
```

Avantage sécurité :

- réduction des risques de fuite de données
- authentification centralisée



Optimisation :

- Moins de requêtes envoyées vers la base de données
- On a plus de performance
- Meilleure expérience pour utilisateur

✓ Avantages :

- Plus sécurisé (pas uniquement visuelle)
- protection contre la modification des URL
- centralisation des permissions
- maintenance facilitée

Vérification systématique des droits utilisateur

Avant chaque action (par exemple cliquer sur un bouton de validation , transférer un colis, confirmer, retirer un colis), vérifier le rôle de l'utilisateur obligatoirement.

Pour empêcher qu'un utilisateur non autorisé qui clique sur des boutons alors qu'il devra pas sensé avoir les permissions.

Par exemple :

- Seul le service postal IUT peut marquer un colis comme retiré
- Un département ne peut que consulter ses colis

Et justement tout ça devra être vérifié.